

实验名称 实验5 复杂网络的节点重要性及相似性分析

一、实验内容及结果

- 1、给定网络 1tux400.csv，通过度中心性、介数中心性、接近中心性、特征向量中心性、PageRank 值这 5 个指标，找出排在前 15%的公共的节点。
- 2、给定网络 1tux400.csv，计算两两节点的相似性，并写入.csv 文件。

源代码：

TASK1:

导入网络，计算前 15%的具体个数：

```
: import networkx as nx
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import csv

df= pd.read_csv("D:/Netdata/ex3/1tux400.csv")
G=nx.from_pandas_edgelist(df,'source','target',weight='weight',create_using=nx.Graph())
num_top_nodes = int(len(G) * 0.15)
print("前15%节点个数:",num_top_nodes)
# nx.draw(G, node_size=50, with_labels=True)
# plt.show()
```

前15%节点个数： 45

五种指标（不考虑权重）：

```
dc= nx.degree_centrality(G)
sorted_dc=sorted(dc.items(), key=lambda x:x[1], reverse=True)
# print("度中心性前15名:",sorted_dc[:15])
top_dc = set([node for node, _ in sorted_dc[:num_top_nodes]])
```

```
bc= nx.betweenness_centrality(G)
sorted_bc=sorted(bc.items(), key=lambda x:x[1], reverse=True)
# print("介数中心性前15名:",sorted_dc[:15])
top_bc = set([node for node, _ in sorted_bc[:num_top_nodes]])
```

```
cc = nx.closeness centrality(G)
sorted_cc = sorted(cc.items(), key=lambda x:x[1], reverse=True)
# print("接近中心性前15名:", sorted_cc[:15])
top_cc = set([node for node, _ in sorted_cc[:num_top_nodes]])
```

```
ec = nx.eigenvector centrality(G, max_iter=200, tol=1e-06)
sorted_ec = sorted(ec.items(), key=lambda x:x[1], reverse=True)
# print("特征向量中心性前15名:", sorted_ec[:15])
top_ec = set([node for node, _ in sorted_ec[:num_top_nodes]])
```

```
pr = nx.pagerank(G, alpha=0.85, max_iter=200, tol=1e-06)
sorted_pr = sorted(pr.items(), key=lambda x:x[1], reverse=True)
# print("pagerank 中心性指标前15名:", sorted_dc[:15])
top_pr = set([node for node, _ in sorted_pr[:num_top_nodes]])
```

计算前 15%公共节点（不考虑权重）：

```
common_top_nodes = top_dc.intersection(top_dc, top_bc, top_cc, top_ec, top_pr)
print("所有指标的前 15% 共同节点:", common_top_nodes)
```

所有指标的前 15% 共同节点: {'TYR170', 'TYR119', 'ARG81', 'TRP126'}

五种指标（考虑权重）：

```
def weighted_degree centrality(G):
    centrality = {}
    for node in G:
        weighted_degree = sum(G[node][neighbor]['weight'] for neighbor in G[node])
        centrality[node] = weighted_degree

    n = len(G) - 1.0
    for node, degree in centrality.items():
        centrality[node] = degree / n
    return centrality

def weighted_closeness centrality(G):
    closeness = {}
    for node in G:
        total_distance = 0
        for target_node in G.nodes():
            if node != target_node:
                try:
                    shortest_path_length = nx.shortest_path_length(G, source=node, target=target_node, weight='weight')
                    total_distance += shortest_path_length
                except nx.NetworkXNoPath:
                    continue

        if total_distance > 0:
            closeness[node] = 1 / total_distance
        else:
            closeness[node] = 0
    return closeness
```

```
w_dc = weighted_degree_centrality(G)
sorted_w_dc=sorted(w_dc.items(), key=lambda x:x[1], reverse=True)
top_w_dc = set([node for node, _ in sorted_w_dc[:num_top_nodes]])
```

```
w_bc = nx.betweenness_centrality(G, weight='weight')
sorted_w_bc=sorted(w_bc.items(), key=lambda x:x[1], reverse=True)
top_w_bc = set([node for node, _ in sorted_w_bc[:num_top_nodes]])
```

```
w_cc= weighted_degree_centrality(G)
sorted_w_cc=sorted(w_cc.items(), key=lambda x:x[1], reverse=True)
top_w_cc = set([node for node, _ in sorted_w_cc[:num_top_nodes]])
```

```
w_ec= nx.eigenvector_centrality(G,max_iter=200, tol=1e-06,weight='weight')
sorted_w_ec=sorted(w_ec.items(), key=lambda x:x[1], reverse=True)
top_w_ec = set([node for node, _ in sorted_w_ec[:num_top_nodes]])
```

```
w_pr= nx.pagerank(G, alpha=0.85, max_iter=200, tol=1e-06, weight="weight")
sorted_w_pr=sorted(w_pr.items(), key=lambda x:x[1], reverse=True)
top_w_pr = set([node for node, _ in sorted_w_pr[:num_top_nodes]])
```

介数中心性、特征向量中心性、pagerank 算法在 networkx 的对应库函数中有考虑权重的设置，而度中心性、接近中心性需要自己定义考虑权重情况下的函数。

计算前 15%公共节点（考虑权重）：

```
w_common_top_nodes = top_dc.intersection(top_w_dc, top_w_bc, top_w_cc, top_w_ec, top_w_pr)
print("所有指标的前 15% 共同节点:", w_common_top_nodes)
```

所有指标的前 15% 共同节点: {'GLN58', 'TRP51'}

TASK2:

```
CN=nx.common_neighbor_centrality(G,sorted(list(G.edges)))
# print("共同邻居中心性是: \n")
# for u,v,p in CN:
#     print(f"({u},{v}) -> {p:.6f}")
```

```
JC=nx.jaccard_coefficient(G,sorted(list(G.edges)))
# print("Jaccard指标是: \n")
# for u,v,p in JC:
#     print(f"({u},{v}) -> {p:.6f}")
```

```
PA=nx.preferential_attachment(G,sorted(list(G.edges)))
# print("优先链接指标: \n")
# for u,v,p in PA:
#     print(f"({u},{v}) -> {p:.6f}")
```

```
AA=nx.adamic_adar_index(G,sorted(list(G.edges)))
# print("Academic-Adar指标是: \n")
# for u,v,p in AA:
#     print(f"({u},{v}) -> {p:.6f}")
```

```
RA=nx.resource_allocation_index(G,sorted(list(G.edges)))
# print("资源分配指标是: \n")
# for u,v,p in RA:
#     print(f"({u},{v}) -> {p:.6f}")
```

```

import csv

CN = nx.common_neighbor_centrality(G, sorted(list(G.edges)))
JC = nx.jaccard_coefficient(G, sorted(list(G.edges)))
PA=nx.preferential_attachment(G,sorted(list(G.edges)))
AA=nx.adamic_adar_index(G,sorted(list(G.edges)))
RA=nx.resource_allocation_index(G,sorted(list(G.edges)))

with open('similarity.csv', 'w', newline='') as csvfile:
    fieldnames = ['node1', 'node2', 'CN', 'JC', 'PA', 'AA', 'RA']
    writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
    writer.writeheader()

    for (u, v, p0), (_, _, p1), (_, _, p2), (_, _, p3), (_, _, p4) in zip(CN, JC, PA, AA, RA):
        writer.writerow({'node1': u, 'node2': v, 'CN': p0, 'JC': p1, 'PA': p2, 'AA': p3, 'RA': p4})

```

运行结果（截图）：

TASK1:

不考虑网络权重：

所有指标的前 15% 共同节点：{'TYR170', 'TYR119', 'ARG81', 'TRP126'}

在以上五种网络指标中，当默认路径权重为 1 时，公共节点有 'TYR170'，'TYR119'，'ARG81'，'TRP126'。

考虑权重：

所有指标的前 15% 共同节点：{'GLN58', 'TRP51'}

在以上五种网络指标中，当考虑网络自带的路径权重时，公共节点有 'GLN58'，'TRP51'。

TASK2:

输出结果 csv 文件:

1	node1	node2	CN	JC	PA	AA	RA
2	ALA1	ALA2	61	0.25	6	0.621335	0.2
3	ALA1	LEU301	61	0.166667	10	0.910239	0.333333
4	ALA12	ARG13	61.8	0.2	35	1.155642	0.35
5	ALA12	ASP11	61	0.142857	15	0.721348	0.25
6	ALA12	GLY14	61.8	0.285714	20	1.424138	0.47619
7	ALA125	GLY82	61	0.1	30	0.389871	0.076923
8	ALA125	TRP126	62.6	0.1875	78	1.489924	0.4
9	ALA125	TYR170	61	0.066667	60	0.389871	0.076923
10	ALA132	ARG140	61.8	0.2	35	0.883328	0.208333
11	ALA132	LEU139	61.8	0.153846	56	1.102233	0.325
12	ALA132	PHE133	61.8	0.153846	56	0.961797	0.25
13	ALA156	ALA160	61	0.0625	70	0.45512	0.111111
14	ALA156	ILE153	61.8	0.153846	56	0.792301	0.160256
15	ALA156	ILE155	61.8	0.153846	56	0.844991	0.188034
16	ALA156	PHE157	61.8	0.117647	84	0.936018	0.236111
17	ALA156	THR159	62.6	0.230769	63	1.317622	0.308333
18	ALA160	ALA162	63.4	0.363636	50	2.052239	0.569444
19	ALA160	ALA163	63.4	0.333333	60	1.974385	0.52702
20	ALA160	ARG161	61.8	0.142857	60	1.102233	0.325
21	ALA160	ASP164	62.6	0.2	80	1.596478	0.457576
22	ALA160	LEU169	62.6	0.2	80	1.430263	0.368687
23	ALA162	ARG161	61	0.1	30	0.434294	0.1
24	ALA163	ALA162	62.6	0.375	30	1.370312	0.336111
956	VAL6	VAL249	61	0.071429	54	0.434294	0.1
957	VAL70	GLN74	61.8	0.153846	54	1.076455	0.311111
958	VAL70	ILE123	61.8	0.117647	90	0.889414	0.211111
959	VAL70	TYR119	62.6	0.1875	90	1.447525	0.377778
960	VAL86	ALA156	61	0.071429	56	0.389871	0.076923
961	VAL86	ASN130	61.8	0.125	80	0.857549	0.194444
962	VAL86	HIS88	62.6	0.176471	96	1.382276	0.34359
963	VAL86	ILE112	61	0.055556	88	0.417032	0.090909
964	VAL86	TRP87	61	0.076923	48	0.40243	0.083333
965	VAL86	TYR152	61.8	0.105263	104	0.916328	0.22619
966	VAL95	ILE98	61	0.083333	42	0.434294	0.1
967	VAL95	LEU104	61	0.0625	70	0.558111	0.166667
968	VAL95	VAL143	60.2	0	35	0	0
969	VAL95	VAL96	61	0.111111	21	0.480898	0.125
970	VAL96	SER97	60.2	0	9	0	0

由于是对同一网络计算五种两两节点相似性，五个指标共享 Csv 表格中的 ‘node1’ , ‘node2’。